Lab1 稀疏一元多项式实验报告

1. 问题描述

本实验的目标是使用链表来实现 **稀疏一元多项式**。稀疏多项式是指大多数项的系数为零，仅存储系数不为零的项。主要功能是高效地插入、合并多项式项，并实现对一元稀疏多项式的几项基本操作。

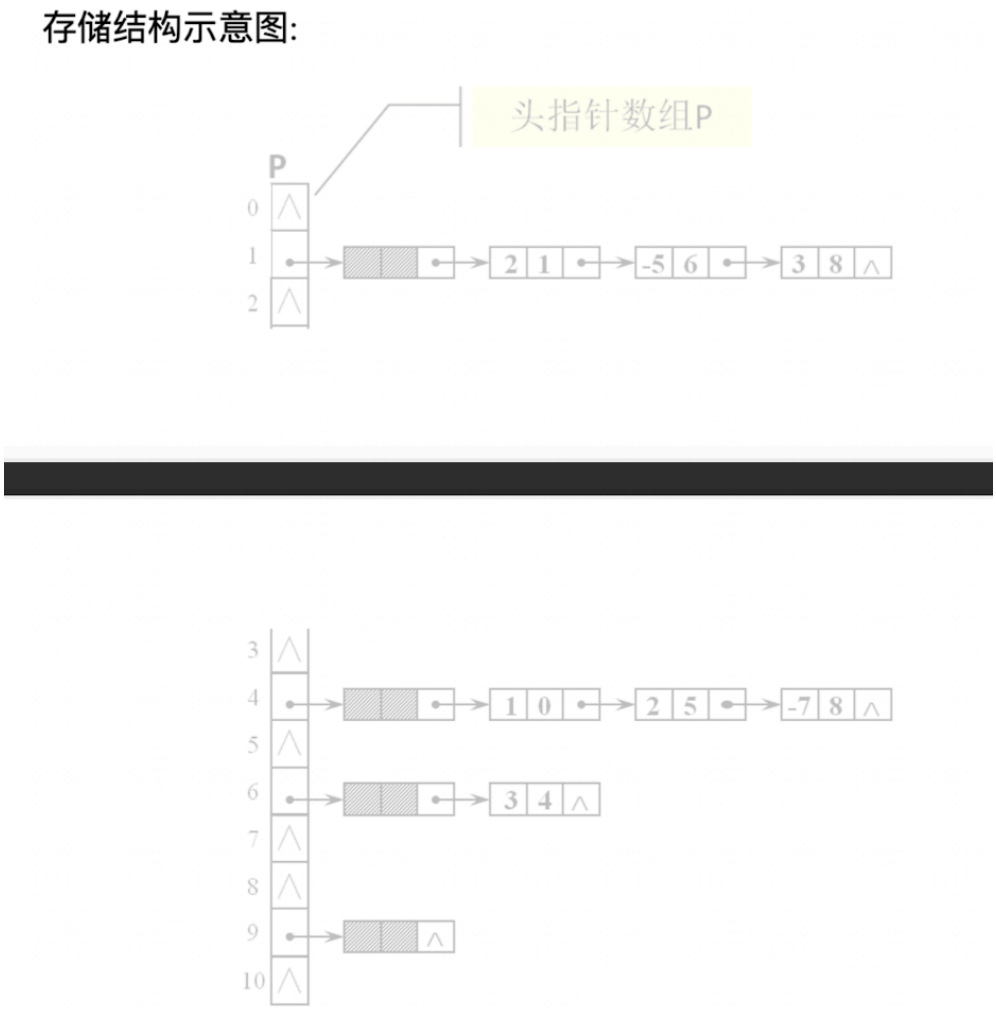
- **输入并创建多项式**（按指数升序或降序排列，系数浮点型，指数整型）
- **输出多项式**，项数+每项系数指数（按指数升序或降序排列）
- **求和** :多项式相加
- **求差** :多项式相减
- **求值**
- **销毁**
- **清空**
- **修改**（1插入新的结点、2删除已有结点、3修改已有结点的系数和指数）
- **微分**（N阶导数）
- **不定积分**
- **定积分**

---

2. 算法描述

(1) 数据结构描述

- **逻辑结构**：使用链表来表示多项式，每个节点存储多项式的一项，包括该项的系数和指数，按指数从大到小的顺序排列。并且将链表表头都存放在指针数组中。
- **存储结构**：
  - `Pitem` 结构体用于表示多项式中的一项，包括：
    - `xishu`：系数 （float）
    - `exp`：指数 （float： 考虑到 后面不定积分中说不定有 lnx 的输出 （把lnx标记为 -1.5次））
    - `next`：指向下一项的指针
  - `Phead` 结构体用于表示多项式的头节点，包括：
    - `number`：多项式项的数量 （int）
    - `next`：指向多项式第一个节点的指针
  - 用 `Phead* P[N] = { NULL }`：定义指针数组，指向各个多项式的头节点以达成如要求所示的存储结构：



关键函数：

- `creatPhead`：初始化一个空的多项式（链表头）。
- `creatPitem`：创建一个新项，包含给定的系数和指数。
- `Pinsert`：将一项插入到多项式中，按指数递减顺序插入（可实现不按顺序插入），若已有相同指数的项则合并系数。

(2) 程序结构

- **函数原型**：
  - `Phead* creatPhead()`：初始化多项式头节点。
  - `Pitem* creatPitem(float xishu, float exp)`：创建多项式的一项。
  - `Pinsert(Phead* head, float xishu, float exp)`：将一项插入到多项式中，若遇到相同指数的项则合并系数。
  - `Pmodify(Phead *head, float old_exp, float new_xishu, float new_exp)`：修改目标项目指数、系数
  - `fixP(Phead *head)`：修改链表 插入、删除、修改节点的系数&指数
  - `creatP(Phead* head )`：多项式建立函数（插入法建立）

- `displayP(Phead *head)`：多项式输出
- `Pvalue(Phead *head, float x)`：多项式求值函数
- `clearP(Phead* head)`： 多项式清空+free空间
- `Phead* addP(Phead* head1, Phead* head2)`： 多项式加法
- `Phead* minusP(Phead* head1, Phead* head2)`： 多项式减法
- `Pderivative(Phead* head)`： 微分
- `Pintegral(Phead *head)`：不定积分
- `float Pdefinite_integrals(Phead* head, float x1, float x2)`：定积分

- **主函数**：
  - 在主函数中， 声明指针数组 `Phead* P[N] = { NULL }` 存放各个后续建立的链表的表头。
  - 用switch - case 在菜单界面选择不同的函数对多项式进行操作。

---

3. 调试与分析

- **测试数据**：
  测试使用了多种多项式组合，例如：
  - 多项式 1: (3x^5 + 2x^3 - 4x^2 + 6)
  - 多项式 2: (5x^4 - 3x^3 + x^2 - 9)
  - .....
- **遇到的问题 & 解决方法**：
  1. **内存泄漏**：在合并相同指数的项时，未释放多余节点，导致内存泄漏问题。为此，添加了 `free(newitem)` 来释放不需要的节点。
  2. **边界情况**：
     - `Pinsert`插入系数为 0 的项时，必须跳过这些项，防止它们进入链表。
     - `displayP`、`Pintegral`：多项式输出时，若要输出不定积分的结果，可能会遇到 lnx 的输出，此时通过在 `Pintegral` 中把 x^-1 项的积分结果的指数储存为 -1.5 加以区分和针对性输出。
     - `Pderivative`：求微分
       - 根据助教哥哥的要求， 在微分结束时， 应输出 0 而不是 “多项式为空”，由此更改输出逻辑为，每一轮微分后，判断一次多项式是否为 0 ，为 0 则直接输出，跳过`displayP`（它会在多项式没有内容（每一项均为 0 ）时输出 ”多项式为空喵～“。
       - 对于输入的过大的求导次数， 函数会针对性输出，“该多项式在第%d次求微分前时已为 0 喵～ 无法继续求微分喵～”， 由此保证了程序的健壮性。
  3. **重复指数项的插入：**
     - 问题：当插入与现有项指数相同的项时，程序应将系数合并。在调试初期，因索引处理不当，合并出现错误。
     - 解决方法：修正查找相同指数项的逻辑，在合并前正确定位到相同指数的位置。

4. 时空复杂度分析

- **时间复杂度：**
  - **插入操作：** 插入操作需要遍历链表，找到适合插入的位置。在最坏情况下，需要遍历所有项，因此时间复杂度为 O(n)，其中 n 是多项式项的数量。
  - **删除操作：** 删除操作同样需要遍历链表，因此时间复杂度也是 O(n)。
  - **修改操作：** 修改操作包括删除旧项和插入新项，因此整体时间复杂度为 O(n)。
  - **求导和积分：** 这些操作需要遍历多项式的所有项，时间复杂度为 O(n)。
- **空间复杂度：**
  - 空间复杂度为 O(n)，其中 n 是多项式的项数，因为每一项都存储在链表的一个节点中。

5.测试结果及分析

- **测试存储 & 输出链表**：

```
================================
            多项式计算器
================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
================================
请输入您的选择喵～: 1
您选择了输入多项式喵～
输入多项式的每一项 (系数和指数), 输入 'n' 停止:
输入系数: 3
输入指数: 5
继续输入下一项吗? (y/n): y
输入系数: 2
输入指数: 3
继续输入下一项吗? (y/n): y
输入系数: -4
输入指数: 2
继续输入下一项吗? (y/n): y
输入系数: 6
输入指数: 0
继续输入下一项吗? (y/n): n
继续输入下一个多项式吗? (y/n): y
输入多项式的每一项 (系数和指数), 输入 'n' 停止:
输入系数: 5
输入指数: 4
继续输入下一项吗? (y/n): y
输入系数: -3
输入指数: 3
继续输入下一项吗? (y/n): y
输入系数: 1
输入指数: 2
继续输入下一项吗? (y/n): y
输入系数: -9
输入指数: 0
继续输入下一项吗? (y/n): n
继续输入下一个多项式吗? (y/n): n
```

```
==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
            3.000x^5 + 2.000x^3 + -4.000x^2 + 6.000
            多项式共有4项
多项式P2为喵～:
            5.000x^4 + -3.000x^3 + x^2 + -9.000
            多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～
```

- **测试多项式加法（P1 + P2， 结果覆盖 P1）**：

```
==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 3
您选择了多项式加法喵～
请输入两个个多项式P1、P2喵～:
如果您想使用之前已经使用过的多项式, 请输入 1 喵, 否则请输入 0 喵~: 1
请选择要相加的两条多项式序号(中间用空格隔开): 1 2
           3.000x^5 + 5.000x^4 + (-x^3) + -3.000x^2 + -3.000
           多项式共有5项
```

```
==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
            3.000x^5 + 5.000x^4 + (-x^3) + -3.000x^2 + -3.000
            多项式共有5项
多项式P2为喵～:
            5.000x^4 + -3.000x^3 + x^2 + -9.000
            多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～
```

- **测试多项式减法（P1 - P2， 结果覆盖 P2）**

```
==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 4
您选择了多项式减法喵～
请输入两个个多项式P1、P2 (P1-P2) 喵～:
如果您想使用之前已经使用过的多项式, 请输入 1 喵( 先前输入的将被视为被减数 ), 否则请输入 0 喵~: 1
请选择要相减的两条多项式序号(中间用空格隔开): 1 2
            3.000x^5 + 2.000x^3 + -4.000x^2 + 6.000
            多项式共有4项

==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
            3.000x^5 + 2.000x^3 + -4.000x^2 + 6.000
            多项式共有4项
多项式P2为喵～:
            5.000x^4 + -3.000x^3 + x^2 + -9.000
            多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～
```

- **测试多项式求值 (对 P1 进行求值， x = 2.2）**

```
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
            3.000x^5 + 2.000x^3 + -4.000x^2 + 6.000
            多项式共有4项
多项式P2为喵～:
            5.000x^4 + -3.000x^3 + x^2 + -9.000
            多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～

==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 5
您选择了多项式求值喵～
请您选择要求值的多项式序号喵～: 1
请输入x的值喵～: 2.2
代入后求得的多项式值为 162.544983 喵～
```

- **清空 & 销毁多项式**

```
==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 6
您选择了清空多项式喵～
请您选择要清空的多项式序号喵～: 1
多项式已清空喵～

==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
```

- **修改多项式某节点**:

  添加新节点:

```
==================================
请输入您的选择喵～: 8
请您选择要修改的多项式序号喵～: 1

请选择操作类型:
1. 插入新的节点
2. 删除已有的节点
3. 修改已有节点的系数和指数
4. 退出操作
输入您的选择: 1
请输入要插入的系数喵～: 4
请输入要插入的指数喵～: 7
插入成功!

==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
           4.000x^7 + 3.000x^5 + 2.000x^3 + -4.000x^2 + 6.000
           多项式共有5项
多项式P2为喵～:
           5.000x^4 + -3.000x^3 + x^2 + -9.000
           多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～
```

  删除节点

```
==================================
请输入您的选择喵～: 8
请您选择要修改的多项式序号喵～: 1

请选择操作类型:
1. 插入新的节点
2. 删除已有的节点
3. 修改已有节点的系数和指数
4. 退出操作
输入您的选择: 2
请输入要删除的节点的指数喵～: 7
已经找到目标项目喵～
修改成功喵～

==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
          3.000x^5 + 2.000x^3 + -4.000x^2 + 6.000
          多项式共有4项
多项式P2为喵～:
          5.000x^4 + -3.000x^3 + x^2 + -9.000
          多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～
```

修改节点:

```
==================================
请输入您的选择喵～: 8
请您选择要修改的多项式序号喵～: 1

请选择操作类型:
1. 插入新的节点
2. 删除已有的节点
3. 修改已有节点的系数和指数
4. 退出操作
输入您的选择: 3
请输入要修改的节点的原指数喵～: 5
请输入新的系数喵～: 6
请输入新的指数喵～: 4
已经找到目标项目喵～
修改成功喵～

==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
          6.000x^4 + 2.000x^3 + -4.000x^2 + 6.000
          多项式共有4项
多项式P2为喵～:
          5.000x^4 + -3.000x^3 + x^2 + -9.000
          多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～
```

- **多项式求微分（对 P1 进行测试）**：

```
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
          6.000x^4 + 2.000x^3 + -4.000x^2 + 6.000
          多项式共有4项
多项式P2为喵～:
          5.000x^4 + -3.000x^3 + x^2 + -9.000
          多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～

==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 9
请选择您要求微分的多项式序号喵～: 1
请输入您要求几阶导数: 4
微分结果为:
          144.000
          多项式共有1项
```

```
==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 9
请选择您要求微分的多项式序号喵～: 1
请输入您要求几阶导数: 4
微分结果为:
          144.000
          多项式共有1项

==================================
              多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 9
请选择您要求微分的多项式序号喵～: 1
请输入您要求几阶导数: 2
多项式在第1次求微分后变为0喵～
```

- **多项式求积分（对 P2 进行测试）**：

> 常数项 C 此处用 0.000 表示，可修改（未计入多项式项数中）

```
请输入您的选择喵～: 2
您选择了显示多项式喵～
多项式P1为喵～:
多项式为空喵～
多项式P2为喵～:
          5.000x^4 + -3.000x^3 + x^2 + -9.000
          多项式共有4项
多项式P3为空喵～
多项式P4为空喵～
多项式P5为空喵～
多项式P6为空喵～
多项式P7为空喵～
多项式输出完成喵～

==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 10
请选择您要积分的多项式序号喵～: 2
积分结果为:
          x^5 + -0.750x^4 + 0.333x^3 + -9.000x^1 + 0.000
          多项式共有4项
```

- **多形式定积分（用 P2 进行测试）**：

```
==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 9
请选择您要求微分的多项式序号喵～: 2
请输入您要求几阶导数: 1
微分结果为:
          5.000x^4 + -3.000x^3 + x^2 + -9.000
          多项式共有4项

==================================
            多项式计算器
==================================
1. 输入多项式
2. 显示多项式
3. 多项式加法
4. 多项式减法
5. 多项式求值
6. 清空多项式
7. 销毁多项式
8. 修改多项式某节点
9. 多项式求微分
10. 多项式求不定积分
11. 多项式定积分
0. 退出
==================================
请输入您的选择喵～: 11
请选择您要积分的多项式序号喵～: 2
请输入积分的上限喵～: 5
请输入积分的下限喵～: 3
定积分求得的值为 2488.666748 喵～
```

---

6. 实验体会和收获

通过本次实验，我加深了对使用链表处理稀疏多项式的理解。实现插入、删除和修改操作时，需要特别注意指针的操作和内存管理。此外，在求导和积分操作时，程序需要对多项式项的次幂和系数进行准确的操作和处理。

本次实验的主要收获有：

- 重温了链表知识。
- 通过调试，学习了如何有效避免和解决内存泄漏问题。
- 认识到在实现算法时，处理边界情况和极端输入的重要性。

Lab2 实验报告

实验报告：括号匹配程序设计与实现

1. 问题描述

1.1 设计目标

本实验的目标是设计并实现一个括号匹配的程序，检查给定的表达式中的括号是否正确匹配。要求处理不同类型的括号（圆括号 `()`、方括号 `[]`、大括号 `{}`），并判断其是否按正确的顺序和配对方式出现。

1.2 输入数据

输入数据由一个整数 `N` 和随后的 `N` 行字符串组成，每行字符串表示一个数学或编程表达式，可能包含多种括号，字符串以`@`结束。

例如：

```
3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b)}@
{1+2[c-d(7/9)}@
```

1.3 输出结果

对于每个表达式，输出其括号是否匹配的结果：

- 如果括号正确匹配，输出 "YES"。
- 如果括号不匹配，输出 "NO"。

例如：

```
YES
NO
NO
```

2. 算法描述

2.1 数据结构的描述

在该程序中，我们使用了栈（`Stack`）来处理括号的匹配。栈能够有效地支持括号的后进先出（LIFO）特性，从而判断括号是否匹配。栈的结构包含以下元素：

- **data**: 存储栈元素的数组。
- **top**: 栈顶指针，指向栈顶的元素。
- **buttom**: 栈底指针，初始化为0。

栈的操作有以下几种：

- **push**: 将括号入栈。
- **pop**: 将栈顶元素出栈。
- **isEmpty**: 判断栈是否为空。

```
struct Stack {
    char data[MAX_SIZE];
    int top;
    int buttom;

    Stack() : top(-1), buttom(0) {}

    void push(char c) {
        if (top < MAX_SIZE - 1) {
            data[++top] = c;
        } else {
            std::cerr << "栈已满" << std::endl;
        }
    }

    char pop() {
        if (top >= buttom) {
            return data[top--];
        } else {
            return '\0';
        }
    }

    bool isEmpty() {
        return top < buttom;
    }
};
```

2.2 算法流程

1. **读取输入**：首先读取整数 `N` 表示有 `N` 行表达式，然后逐行读取每个表达式。
2. 括号匹配
   ：对每个表达式进行处理：
   - 遍历每个字符，如果是左括号（`(`, `[`, `{`），则将其入栈。
   - 如果是右括号（`)`, `]`, `}`），则判断栈是否为空，以及栈顶元素是否匹配。
3. **输出结果**：根据括号是否匹配，输出 `YES` 或 `NO`。

2.3 优先级判断

为了支持不同类型括号的优先级比较，我们定义了一个简单的优先级函数：

```
int Priority(char c) {
    if (c == '{') return 3;
    if (c == '[') return 2;
    if (c == '(') return 1;
}
```

在遍历括号时，若发现当前左括号的优先级大于栈顶括号的优先级，则返回 `"NO"`。

2.4 主要函数

`Bracket_pairing` 函数用于判断给定的表达式中的括号是否正确匹配。函数通过栈来处理括号配对，逐个字符进行检查。

```
const char* Bracket_pairing(const char* expression) {
    Stack stack;
    for (int i = 0; expression[i] != '@'; i++) {
        char c = expression[i];

        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
            if (stack.top > 0) {
                char rear = stack.data[stack.top - 1];
                if (Priority(c) > Priority(rear)) {
                    return "NO";
                }
            }
        } else if (c == ')' || c == ']' || c == '}') {
            if (stack.isEmpty()) {
                return "NO";
            }
            char top = stack.pop();

            if ((c == ')' && top != '(') ||
```

```
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) {
                return "NO";
            }
        }
    }
    return stack.isEmpty() ? "YES" : "NO";
}
```

2.5 主函数

主函数负责处理文件的输入输出，并调用`Bracket_pairing`函数进行括号匹配判断。

```
int main() {
    std::ifstream inputFile("input.txt");
    std::ofstream outputFile("output.txt");

    if (!inputFile.is_open()) {
        std::cerr << "无法打开文件!" << std::endl;
    }

    int N;
    inputFile >> N;
    inputFile.ignore();

    for (int i = 0; i < N; i++) {
        char expression[MAX_SIZE];
        inputFile.getline(expression, MAX_SIZE);
        outputFile << Bracket_pairing(expression) << std::endl;
    }

    inputFile.close();
    outputFile.close();

    return 0;
}
```

2.6 全部代码

```
#include<iostream>
#include<fstream>
#include<cstdlib>

#define MAX_SIZE 1000
#define STACK_MAX_SIZE 1000

struct Stack{
```

```
    char data[MAX_SIZE];
    int top;
    int buttom;

    Stack() : top(-1), buttom(0) {}

    // 入栈
    void push(char c) {
        if (top < MAX_SIZE - 1) {
            data[++top] = c;
        } else {
            std::cerr << "栈已满" << std::endl;
        }
    }

    // 出栈
    char pop() {
        if (top >= buttom) {
            return data[top--];
        } else {
            return '\0';
        }
    }

    // 空栈
    bool isEmpty() {
        return top < buttom;
    }
};

int Priority(char c) {
    if (c == '{') return 3;
    if (c == '[') return 2;
    if (c == '(') return 1;
}

const char* Bracket_pairing(const char* expression) {
    Stack stack;
    for (int i = 0; expression[i] != '@'; i++) {
        char c = expression[i];

        // 左括号入栈
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
            if (stack.top > 0) {
                char rear = stack.data[stack.top - 1];
                // 判断括号优先级
                if (Priority(c) > Priority(rear)) {
                    return "NO";
```

```
                }
            }
        } else if (c == ')' || c == ']' || c == '}') {
            // 判断碰到右括号时是否为空栈
            if (stack.isEmpty()) {
                return "NO";
            }
            // 右括号出栈 比较
            char top = stack.pop();

            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) {
                return "NO";
            }

        }
    }

    // 最后为空栈，说明匹配完成
    return stack.isEmpty() ? "YES" : "NO";
}

int main() {
    std::ifstream inputFile("input.txt");
    std::ofstream outputFile("output.txt");

    // 打开失败
    if (!inputFile.is_open()) {
        std::cerr << "Can not open the file! " << std::endl;
    }

    // 读入 N 循环
    int N;
    inputFile >> N;
    inputFile.ignore();

    for (int i = 0; i < N; i++) {
        char expression[MAX_SIZE];
        // 逐行读入
        inputFile.getline(expression, MAX_SIZE);
        // 对应输出
        outputFile << Bracket_pairing(expression) << std::endl;
    }
    // 关闭文件
    inputFile.close();
    outputFile.close();

    return 0;
```

```
}
```

---

3. 调试分析

3.1 测试数据选择

为了验证程序的正确性，选择了多种不同类型的表达式进行测试，确保程序能够正确判断各种括号配对的情况。

测试数据：

```
3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b)}@
{1+2[c-d(7/9)}@
```

预期输出：

```
YES
NO
NO
```

3.2 程序调试中遇到的问题

1. **优先级问题**：在处理优先级时，误判断了不同类型括号的顺序，通过调试逐步修正了优先级比较的逻辑。
2. **判断逻辑问题**：在处理此处的括号配对判断时，

```
        else if (c == ')' || c == ']' || c == '}') {
            // 判断碰到右括号时是否为空栈
            if (stack.isEmpty()) {
                return "NO";
            }
            // 右括号出栈 比较
            char top = stack.pop();

            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) {
                return "NO";
            }

        }
```

我一开始并未考虑到碰到有括号是否要判断是否为空栈，导致怎么也运行不对，后续检查时才发现。

4. 算法的时空分析

4.1 时间复杂度

1. **遍历表达式**：对于每个表达式，程序遍历每个字符一次，时间复杂度为 O(m)，其中 m 是表达式的长度。
2. **栈操作**：栈的操作（`push` 和 `pop`）的时间复杂度为 O(1)。

因此，总时间复杂度为 O(m)，其中 m 是每个表达式的长度。

4.2 空间复杂度

栈的大小最大为表达式中的括号数量，空间复杂度为 O(m)，其中 m 是表达式的长度。

5. 测试结果及分析

5.1 测试数据

输入数据：

```
3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b)}@
{1+2[c-d(7/9)}@
```

输出数据：

```
YES
NO
NO
```

5.2 测试结果分析

程序正确识别了括号配对的情况，对于合法的括号对输出 `YES`，对于不匹配的括号对输出 `NO`。结果符合预期。

6. 实验体会和收获

通过本次实验，我深入理解了栈数据结构的使用，特别是在括号匹配问题中的应用。栈通过先进后出的特性，能够高效地处理括号配对问题。在调试过程中，我意识到边界条件的重要性，确保栈的操作不发生越界，并合理处理不同类型括号的优先级。通过这次实验，我对栈的实现和应用有了更深入的理解，同时提高了自己的调试能力和程序设计能力。

实验报告：行编辑器程序设计与实现

1. 问题描述

1.1 设计目标

本实验的目标是设计并实现一个简单的文本编辑器，支持以下基本操作：

- **Move**: 将光标移动到文本中的指定位置。
- **Insert**: 在光标位置插入指定长度的字符串。
- **Delete**: 删除光标后指定长度的字符。
- **Rotate**: 反转光标后指定长度的字符。
- **Prev**: 光标前移一个字符。
- **Next**: 光标后移一个字符。
- **Get**: 输出当前文本。

1.2 输入数据

输入数据包含多个操作命令。每条命令由命令类型和必要的参数组成，格式如下：

- **Move k**：将光标移动到第k个字符之后。
- **Insert n S**：在光标位置插入一个长度为n的字符串S。
- **Delete n**：删除光标后n个字符。
- **Rotate n**：反转光标后n个字符。
- **Prev**：将光标前移一个字符。
- **Next**：将光标后移一个字符。
- **Get**：输出当前文本。

1.3 输出结果

输出文件记录了每次执行`Get`命令时的文本内容。每次调用`Get`时，文本中的内容将被写入文件中，且每次文本后都换行。

2. 算法描述

2.1 数据结构的描述

在该程序中，我们使用了一个**数组**来存储文本内容。光标的位置和文本的当前长度是程序的关键变量。具体数据结构如下：

- **text**: 一个字符数组，用于存储当前文本内容。最大长度为`MAX_TEXT_SIZE`（1000）。

- **cursor**: 一个整数，表示光标的位置。
- **length**: 一个整数，表示当前文本的长度。

```
const int MAX_TEXT_SIZE = 1000; // 最大文本长度
const int MAX_COMMAND_SIZE = 50; // 最大命令长度

class TextEditor {
private:
    char text[MAX_TEXT_SIZE];  // 文本内容
    int cursor;  // 光标位置
    int length;  // 文本当前长度
public:
    // 构造函数
    TextEditor();
    void move(int k);
    void insert(int n, const char* s);
    void deleteChars(int n);
    void rotate(int n);
    void get(std::ofstream& outputFile);
    void prev();
    void next();
};
```

2.2 程序结构的描述

程序的主要功能通过`TextEditor`类实现，类中包含多个成员函数来处理不同的文本编辑命令。

1. **move(int k)**: 将光标移动到第k个字符之后，如果k为0，则将光标移到文本的开头。
2. **insert(int n, const char* s)**: 在光标位置插入一个长度为n的字符串。
3. **deleteChars(int n)**: 删除光标后n个字符。
4. **rotate(int n)**: 反转光标后n个字符。
5. **get(std::ofstream& outputFile)**: 输出当前文本到文件。
6. **prev()**: 光标前移一个字符。
7. **next()**: 光标后移一个字符。

```
bool compareCommand(const char* inputCommand, const char* expectedCommand);
int main() {
    std::ifstream inputFile("input.txt");
    std::ofstream outputFile("output.txt");

    // 读取命令并执行相应操作
}
```

2.3 命令比较

为了判断命令是否匹配，我们实现了一个辅助函数`compareCommand`，通过逐字符比较输入命令和预期命令是否一致。

```
bool compareCommand(const char* inputCommand, const char* expectedCommand);
```

2.4 主函数流程

主函数负责读取输入命令，创建`TextEditor`对象并根据不同的命令调用对应的成员函数。命令格式如下：

1. 读取命令类型。
2. 根据命令类型读取相应参数并调用编辑器方法。
3. 执行`Get`命令时，输出当前文本到文件。

```
int main() {
    std::ifstream inputFile("input.txt");
    std::ofstream outputFile("output.txt");

    int numCommands;
    inputFile >> numCommands;
    inputFile.ignore(); // 忽略换行符

    TextEditor editor;
    for (int i = 0; i < numCommands; ++i) {
        char command[MAX_COMMAND_SIZE];
        inputFile >> command;

        if (compareCommand(command, "Move")) {
            int k;
            inputFile >> k;
            editor.move(k);
        } else if (compareCommand(command, "Insert")) {
            int n;
            char s[MAX_COMMAND_SIZE];
            inputFile >> n;
            inputFile.ignore(); // 忽略空格
            for (int j = 0; j < n; ++j) {
                s[j] = inputFile.get(); // 逐个读取字符
            }
            s[n] = '\0'; // 添加字符串结束符
            editor.insert(n, s);
        } else if (compareCommand(command, "Delete")) {
            int n;
            inputFile >> n;
            editor.deleteChars(n);
        } else if (compareCommand(command, "Rotate")) {
            int n;
```

```
            inputFile >> n;
            editor.rotate(n);
        } else if (compareCommand(command, "Get")) {
            editor.get(outputFile);
        } else if (compareCommand(command, "Prev")) {
            editor.prev();
        } else if (compareCommand(command, "Next")) {
            editor.next();
        }
    }

    inputFile.close();
    outputFile.close();
    return 0;
}
```

2.5 全部代码

```
#include <iostream>
#include <fstream>

const int MAX_TEXT_SIZE = 1000;  // 假设最大文本长度为1000
const int MAX_COMMAND_SIZE = 50; // 假设命令字符串的最大长度

class TextEditor {
private:
    char text[MAX_TEXT_SIZE];
    int cursor;
    int length; // 当前文本长度

public:
    TextEditor(): cursor(0), length(0) {
        text[0] = '\0';
    }

    // 将光标移动到第 k 个字符之后，如果 k=0 则将光标移到文本第一个字符之前
    void move(int k) {
        if (k >= 0 && k <= length) {
            cursor = k;
        } else if (k > length) {
            cursor = length;
        } else {
            cursor = 0;
        }
    }

    // 在光标后插入长度为 n 的字符串 S，光标位置不变，n≥1
```

```
void insert(int n, const char* s) {
    if (length + n > MAX_TEXT_SIZE) {
        return;
    }
    // 光标后内容向后移动n个单位
    for (int i = length - 1; i >= cursor; --i) {
        text[i + n] = text[i];
    }
    // 向光标后插入s
    for (int i = 0; i < n; ++i) {
        text[cursor + i] = s[i];
    }
    length += n;
    text[length] = '\0';
}

// 删除光标后的 n 个字符，光标位置不变，n≥1
void deleteChars(int n) {
    if (n <= 0 || cursor >= length) return;
    int deleteCount = (cursor + n > length) ? length - cursor : n;
    for (int i = cursor; i + deleteCount < length; ++i) {
        text[i] = text[i + deleteCount];
    }
    length -= deleteCount;
    text[length] = '\0';
}

// 反转光标后的 n 个字符，光标位置不变，n≥1
void rotate(int n) {
    if (n <= 0 || cursor + n > length) return;
    for (int i = 0; i < n / 2; ++i) {
        char temp = text[cursor + i];
        text[cursor + i] = text[cursor + n - i - 1];
        text[cursor + n - i - 1] = temp;
    }
}

// 输出此时程序内保存的文本
void get(std::ofstream& outputFile) {
    for (int i = 0; i < length; ++i) {
        outputFile.put(text[i]);
    }
    outputFile.put('\n');
}

// 光标前移一个字符
void prev() {
    if (cursor > 0) {
        cursor--;
```

```
        }
    }

    // 光标后移一个字符
    void next() {
        if (cursor < length) {
            cursor++;
        }
    }
};

bool compareCommand(const char* inputCommand, const char* expectedCommand) {
    int i = 0;
    while (inputCommand[i] != '\0' && expectedCommand[i] != '\0') {
        if (inputCommand[i] != expectedCommand[i]) {
            return false;
        }
        ++i;
    }
    return inputCommand[i] == '\0' && expectedCommand[i] == '\0';
}

int main() {
    std::ifstream inputFile("input.txt");
    std::ofstream outputFile("output.txt");

    int numCommands;
    inputFile >> numCommands;
    inputFile.ignore(); // 忽略换行符

    TextEditor editor;
    for (int i = 0; i < numCommands; ++i) {
        char command[MAX_COMMAND_SIZE];
        inputFile >> command;

        if (compareCommand(command, "Move")) {
            int k;
            inputFile >> k;
            editor.move(k);
        } else if (compareCommand(command, "Insert")) {
            int n;
            char s[MAX_COMMAND_SIZE];
            inputFile >> n;
            inputFile.ignore(); // 忽略空格
            for (int j = 0; j < n; ++j) {
                s[j] = inputFile.get(); // 逐个读取字符
            }
            s[n] = '\0'; // 添加字符串结束符
            editor.insert(n, s);
```

```
        } else if (compareCommand(command, "Delete")) {
            int n;
            inputFile >> n;
            editor.deleteChars(n);
        } else if (compareCommand(command, "Rotate")) {
            int n;
            inputFile >> n;
            editor.rotate(n);
        } else if (compareCommand(command, "Get")) {
            editor.get(outputFile);
        } else if (compareCommand(command, "Prev")) {
            editor.prev();
        } else if (compareCommand(command, "Next")) {
            editor.next();
        }
    }

    inputFile.close();
    outputFile.close();
    return 0;
}
```

---

3. 调试分析

3.1 测试数据选择

测试数据的选择应该涵盖所有可能的操作，确保每种功能都能正常工作。测试数据包括：

1. 光标移动：测试`Move`命令，确保光标可以正确移动到指定位置。
2. 字符插入：测试`Insert`命令，确保字符串能够正确插入到光标位置。
3. 字符删除：测试`Delete`命令，确保删除操作能按预期工作。
4. 字符旋转：测试`Rotate`命令，确保文本内容能被正确反转。
5. 输出：通过`Get`命令，验证输出的文本是否符合预期。

3.2 程序调试中遇到的问题

在调试过程中，最常见的问题包括：

- **光标超出范围**：例如在执行`move`或`delete`命令时，光标可能会越界。通过增加检查条件来解决这个问题。
- **字符插入超出文本范围**：插入的字符串长度可能超出了预设的最大文本长度。解决方法是加入对最大长度的检查。

4. 算法的时空分析

4.1 时间复杂度

1. **move(k)**: 时间复杂度为O(1)，因为移动光标只是修改一个整数值。
2. **insert(n, s)**: 时间复杂度为O(n)，需要移动数组中的元素来腾出空间并插入新字符。
3. **deleteChars(n)**: 时间复杂度为O(n)，需要移动数组中的元素来删除字符。
4. **rotate(n)**: 时间复杂度为O(n)，需要遍历指定的字符段并交换它们的位置。
5. **get(outputFile)**: 时间复杂度为O(m)，其中m为文本的当前长度。

4.2 空间复杂度

程序的空间复杂度主要由`text`数组决定，空间复杂度为O(MAX_TEXT_SIZE)，即O(1000)。

5. 测试结果及分析

5.1 测试数据

测试数据包括一系列`Move`、`Insert`、`Delete`、`Rotate`等命令。每次执行`Get`命令时，程序应输出当前的文本内容。

输入数据：

```
12
Insert 13
Balanced eert
Get
Move 2
Delete 5
Get
Next
Insert 6
editor
Move 0
Get
Move 10
Rotate 4
Get
```

输出数据：

```
Balanced eert
Bad eert
Badeditor eert
Badeditor tree
```

5.2 测试结果分析

测试结果表明，程序能够正确处理所有操作，包括光标移动、字符串插入、字符删除和旋转操作。`Get`命令输出了正确的文本内容。

6. 实验体会和收获

通过本次实验，我更深入理解了文本编辑器的基本操作和数据结构的设计。在实际编程中，细节问题（如边界条件的处理）对程序的正确性至关重要。通过调试和多次测试，我能够更好地把握程序的执行流程，提高了自己的编程能力和调试技巧。

实验报告：银行业务模拟程序设计与实现

1. 问题描述

1.1 设计目标

本实验旨在设计一个银行业务的模拟程序，模拟客户在银行进行交易的过程。程序模拟银行的队列系统，客户根据自己的需求排队等候。每个客户在到达银行后，会根据自己的存款需求和银行的储备金进行交易。如果银行储备金足够满足客户需求，客户立即进行交易并离开；如果储备金不足，客户将被放入二队，等待再次处理。每位客户在银行的等待时间也需要被记录，最终输出所有客户的等待时间。

1.2 输入数据

输入数据包括：

- `N`：客户数量
- `total`：银行初始储备金
- `close_time`：银行的关闭时间（模拟时间）
- `average_time`：每个客户交易所需的时间（每个客户处理一次的时间间隔）

每个客户的记录包括：

- `amount`：客户需求的金额
- `time`：客户到达银行的时间

示例输入数据格式：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

1.3 输出数据

输出数据包含每个客户的等待时间，以及所有客户的平均等待时间。

例如：

```
0
590
30
0
155
```

表示每个客户的等待时间和所有客户的平均等待时间。

2. 算法描述

2.1 数据结构的描述

在这个模拟程序中，我们设计了一个 `Customer` 结构体和一个 `Queue` 类来表示客户和队列：

1. **Customer 结构体**：
   - `amount`：客户的需求金额。
   - `time`：客户到达银行的时间。
   - `wait_time`：客户在银行的等待时间。
2. **Queue 类**：
   - 使用循环队列来存储客户，采用动态内存分配的方式实现。
   - `quene`：客户队列的指针数组，用于存储客户对象。
   - `front`：队列的队首位置。
   - `rear`：队列的队尾位置。
   - `size`：队列当前的大小。
   - `capacity`：队列的最大容量。

队列操作包括：

- `enquene()`：将客户加入队列。
- `outquene()`：从队列中取出客户。
- `peek()`：查看队首客户。

```
struct Quene {
    Customer **quene; // 存储客户的指针数组
    int front;        // 队首
    int rear;         // 队尾
    int size;         // 当前的队列大小
    int capacity;     // 队列容量
```

```
    // 初始化队列
    Quene(int num) {
        capacity = num;
        quene = (Customer**)malloc(capacity * sizeof(Customer*)); // 分配存储Customer指针
的数组
        front = 0;
        rear = -1;
        size = 0;
    }

    // 判断队列为空
    bool isEmpty() {
        return size == 0;
    }

    // 判断队列为满
    bool isFull() {
        return size == capacity;
    }

    // 从队尾入队
    void enquene(Customer *c) {
        if (isFull()) {
            return;
        }
        rear = (rear + 1) % capacity;
        quene[rear] = c;
        size++;
    }

    // 从队首出队
    Customer *outquene() {
        if (isEmpty()) {
            return nullptr;
        }
        Customer *c = quene[front];
        front = (front + 1) % capacity;
        size--;
        return c;
    }

    // 查看队首元素
    Customer *peek() {
        if (isEmpty()) {
            return nullptr;
        }
        return quene[front];
    }
```

```
};
```

2.2 算法流程

1. **初始化**：首先，读取客户数据和银行初始的储备金。
2. **入队操作**：将所有客户按顺序加入队列 `q1`。
3. **银行处理过程**：
   - 每当有客户到达时，检查当前时间与客户的到达时间，若客户已经到达，则尝试为其办理业务。
   - 如果银行储备金足够满足客户需求，客户出队并更新储备金和等待时间。
   - 如果银行储备金不足，则将客户移至队列 `q2`，等待下一轮处理。
   - 若银行关闭时队列中仍有客户，则直接计算他们的等待时间。
4. **输出结果**：程序输出所有客户的等待时间，并计算并输出平均等待时间。

```
void SimulateBank(int N, int total, int close_time, int average_time, Customer
customers[]) {
    Quene q1(N);  // 初始队列
    Quene q2(N);  // 第二队列
    int Current_Time = 0;  // 当前时间
    int Total_Wait_Time = 0;  // 总等待时间
    int Total_Moeny = total;  // 银行储备金

    // 将客户按顺序入队
    for (int i = 0; i < N; i++) {
        q1.enquene(&customers[i]);
        customers[i].wait_time = 0;
    }

    while (!q1.isEmpty() || !q2.isEmpty()) {
        while (!q1.isEmpty() && Current_Time < close_time) {
            Customer *current_customer1 = q1.peek();
            Current_Time = Current_Time >= current_customer1->time ? Current_Time :
current_customer1->time;

            if (current_customer1->amount + Total_Moeny >= 0) {
                q1.outquene();
                Total_Moeny += current_customer1->amount;
                current_customer1->wait_time = Current_Time - current_customer1->time;
                Current_Time += average_time;
            } else {
                q1.outquene();
                q2.enquene(current_customer1);
            }
        }

        while (!q2.isEmpty()) {
            Customer *current = q2.outquene();
```

```
            current->wait_time = close_time - current->time;
        }
    }
}
```

2.3 主函数

主函数负责读取输入文件、调用 `SimulateBank` 函数进行模拟，并将结果写入输出文件。

```
int main() {
    std::ifstream inputFile("Bank_customer_input.txt");
    if (!inputFile.is_open()) {
        std::cerr << "Cannot open the input file!" << std::endl;
        return 1;
    }

    int N, total, close_time, average_time;
    inputFile >> N >> total >> close_time >> average_time;

    Customer *customers = new Customer[N];
    for (int i = 0; i < N; i++) {
        inputFile >> customers[i].amount >> customers[i].time;
    }

    SimulateBank(N, total, close_time, average_time, customers);

    inputFile.close();
    return 0;
}
```

2.6 全部代码

```
#include<iostream>
#include<fstream>
#include <cstdlib>

struct Customer {
    int amount;     // 金额
    int time;       // 到达时间
    int wait_time; // 等候时间
};

struct Quene {
    Customer **quene; // 储存客户的指针数组
    int front;        // 队首
    int rear;         // 队尾
    int size;         // 当前的队列大小
```

```
    int capacity;     // 队伍容量

    // 初始化队列
    Quene(int num) {
        capacity = num;
        quene = (Customer**)malloc(capacity * sizeof(Customer*)); // 分配存储Customer指针
的数组
        front = 0;
        rear = -1;
        size = 0;
    }

    // 判断队列为空
    bool isEmpty() {
        return size == 0;
    }

    // 判断队列为满
    bool isFull() {
        return size == capacity;
    }

    // 从队尾入队
    void enquene(Customer *c) {
        if (isFull()) {
            return;
        }
        rear = (rear + 1) % capacity;
        quene[rear] = c;
        size++;
    }

    // 从队首出队
    Customer *outquene() {
        if (isEmpty()) {
            return nullptr;
        }
        Customer *c = quene[front];
        front = (front + 1) % capacity;
        size--;
        return c;
    }

    // 查看队首元素
    Customer *peek() {
        if (isEmpty()) {
            return nullptr;
        }
        return quene[front];
```

```
    }
};

// 模拟银行
void SimulateBank(int N, int total, int close_time, int average_time, Customer
customers[]) {
    // 建立两个队列
    Quene q1(N);
    Quene q2(N);

    int Current_Time = 0;      // 目前的时间
    int Total_Wait_Time = 0;   // 所有客户总等待时间
    int Total_Moeny = total;   // 目前银行储备金

    // 将客户按顺序入队
    for (int i = 0; i < N; i++) {
        q1.enquene(&customers[i]);
        customers[i].wait_time = 0;
    }

    while (!q1.isEmpty() || !q2.isEmpty()) {
        // 处理队列 1
        while (!q1.isEmpty() && Current_Time < close_time) {
            Customer *current_customer1 = q1.peek();

            int Fore_Total = Total_Moeny;

            // 更新目前的时间，如果他晚来，就以到达时间为准；早来则以前方处理时间为准
            Current_Time = Current_Time >= current_customer1->time ? Current_Time :
current_customer1->time;

            // 客户需求能直接被满足，直接出队，否则进入队 2
            if (current_customer1->amount + Total_Moeny >= 0) {
                q1.outquene();
                Total_Moeny += current_customer1->amount;

                current_customer1->wait_time = Current_Time - current_customer1->time;
                Current_Time += average_time;
            } else {
                q1.outquene();
                q2.enquene(current_customer1);
            }

            int q2_size = q2.size;
            // 处理队列 2
            for (int i = 0; Total_Moeny > Fore_Total && Current_Time < close_time &&
!q2.isEmpty() && i < q2_size; i++) {
                Customer *current_customer2 = q2.peek();
```

```
                Current_Time = Current_Time >= current_customer2->time ? Current_Time :
current_customer2->time;

                // 客户需求能直接被满足，直接出队，否则重新进入队列 2
                if (current_customer2->amount + Total_Moeny >= 0) {
                    q2.outquene();
                    Total_Moeny += current_customer2->amount;

                    current_customer2->wait_time = Current_Time - current_customer2-
>time;
                    Current_Time += average_time;
                } else {
                    q2.enquene(q2.outquene());
                }
            }
        }

        // 当 Current_Time > close_time 时，若还有在队列 2 中的，直接处理他们的 wait_time
        while (!q2.isEmpty()) {
            Customer *current = q2.outquene();
            current->wait_time = close_time - current->time;
        }
    }

    // 输出结果到文件
    std::ofstream outputFile("Bank_customer_waiting_output.txt");
    if (!outputFile.is_open()) {
        std::cerr << "Cannot open the output file!" << std::endl;
        return;
    }

    for (int i = 0; i < N; i++) {
        outputFile << customers[i].wait_time << std::endl;
        Total_Wait_Time += customers[i].wait_time;
    }
    outputFile << (Total_Wait_Time / N) << std::endl;
    outputFile.close();
}

int main() {
    std::ifstream inputFile("Bank_customer_input.txt");
    if (!inputFile.is_open()) {
        std::cerr << "Cannot open the input file!" << std::endl;
        return 1;
    }

    // 读取客户数据
    int N, total, close_time, average_time;
    inputFile >> N >> total >> close_time >> average_time;
```

```
    Customer *customers = new Customer[N];

    for (int i = 0; i < N; i++) {
        inputFile >> customers[i].amount >> customers[i].time;
    }

    SimulateBank(N, total, close_time, average_time, customers);

    inputFile.close();
    return 0;
}
```

---

3. 调试分析

3.1 测试数据选择

为了验证程序的正确性，我们使用了多个包含不同客户数量和需求金额的数据集进行测试。

测试数据：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

预期输出：

```
0
590
30
0
155
```

3.2 调试问题

1. **队列操作**：初期在队列的出队操作中遇到了一些边界问题，导致客户未能正确处理。通过仔细检查队列的大小、前后指针的更新及循环队列的处理，解决了这一问题。
2. **时间计算**：在处理时间更新时，曾忽略了客户到达时间与当前时间的比较，导致客户等待时间计算错误。通过修正这一部分，确保客户在等待时能够正确地等待。

4. 算法的时空分析

4.1 时间复杂度

1. **入队与出队**：队列的入队和出队操作的时间复杂度为 O(1)。
2. **处理过程**：程序会在每个客户到达时处理一次，时间复杂度为 O(N)，其中 N 是客户的数量。

因此，总时间复杂度为 O(N)。

4.2 空间复杂度

1. **客户数据存储**：客户数据的存储需要 O(N) 空间。
2. **队列存储**：每个队列需要 O(N) 空间。

因此，总空间复杂度为 O(N)。

5. 测试结果及分析

5.1 测试数据

输入数据：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

输出数据：

```
0
590
30
0
155
```

5.2 测试结果分析

程序能够正确模拟银行处理客户的过程。每个客户的等待时间根据其到达时间、银行的储备金和交易时间等因素进行计算。程序的输出符合预期，且能够正确计算客户的平均等待时间。

6. 实验体会和收获

通过本次实验，我深入理解了队列的应用，尤其是在模拟银行排队系统中的作用。栈和队列的基本操作在模拟过程中尤为重要，理解它们的基本操作和优化技巧对程序的效率提升至关重要。在调试过程中，我加强了对时间计算和队列操作的掌握，提升了编程和调试的能力。

Lab3 实验报告

实验报告：Huffman压缩与解压缩程序设计与实现

1. 实验目的

本实验旨在实现一个基于Huffman编码的文件压缩和解压缩程序。Huffman编码是一种常用的无损数据压缩算法，利用字符的频率信息对文本进行编码，从而减少数据所占的存储空间。本实验通过实现Huffman树的构建、编码与解码过程，模拟文件的压缩与解压操作。

2. 实验内容

2.1 任务描述

本程序的主要功能包括：

1. **压缩功能**：读取文本文件，统计字符频率，构建Huffman树，生成Huffman编码表，并根据编码表对文件内容进行压缩。
2. **解压功能**：读取压缩文件，恢复Huffman树，解码文件中的压缩数据，恢复原始文本内容。
3. **用户交互**：程序提供命令行界面，允许用户选择进行文件压缩或解压操作。

2.2 输入输出

输入：

- 压缩操作：输入文件名，程序会将该文件压缩为`.huff`格式的压缩文件。
- 解压操作：输入压缩文件名（`.huff`格式），程序会解压该文件并恢复原始文件内容。

输出：

- 压缩后的文件：以`.huff`为扩展名保存压缩文件。
- 解压后的文件：恢复为原始文件格式。

2.3 功能实现

2.3.1 Huffman树的构建

首先，程序会读取文件中的字符，统计每个字符出现的频率。然后，根据字符频率构建一棵Huffman树。构建过程通过一个最小优先队列（链表实现），每次从队列中取出频率最小的两个节点，将其合并为一个新的节点，并将新的节点插回队列。这个过程直到队列中只剩下一个节点时结束。

```
HuffmanNode* Build_Huffman_Tree(int freq[]) {
    ListNode* head = nullptr;
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) {
            HuffmanNode* node = new HuffmanNode((char)i, freq[i]);
            insert(head, node);
        }
    }

    while(head && head->next) {
        HuffmanNode* left = PopMin(head);
        HuffmanNode* right = PopMin(head);

        int sum = left->freq + right->freq;
        HuffmanNode* combine_node = new HuffmanNode('\0', sum);
        combine_node->left = left;
        combine_node->right = right;

        insert(head,combine_node);
    }

    return PopMin(head);
}
```

2.3.2 Huffman编码

通过对Huffman树的深度优先遍历，生成每个字符的Huffman编码。左子树对应`0`，右子树对应`1`，直到叶子节点。每个字符的编码存储在一个数组中。

```
void encodeHuffmanTree(HuffmanNode* root, string str, string HuffmanCode[]) {
    if (!root) return;
    if (!root->left && !root->right) {
        HuffmanCode[(unsigned char) root->ch] = str;
    }
    else {
        encodeHuffmanTree(root->left, str + "0", HuffmanCode);
        encodeHuffmanTree(root->right, str + "1", HuffmanCode);
    }
}
```

2.3.3 压缩与解压缩

- **压缩**：程序会将文本中的每个字符根据其对应的Huffman编码进行转换，形成一个长的比特串。并将字符的频率信息、比特串的长度以及比特串本身存储到输出文件中。
- **解压缩**：程序会读取压缩文件中的频率表，重建Huffman树，然后根据Huffman树对比特串进行解码，恢复原始文本。

```
void compressFile(const string& inputFile, const string& outputFile) {
    ifstream in(inputFile, ios::binary);
    if (!in) {
        cout << "无法打开文件: " << inputFile << endl;
        return;
    }
    string text((istreambuf_iterator<char>(in)), istreambuf_iterator<char>());
    in.close();

    int freq[256] = {0};
    calculateFrequency(freq, text);

    HuffmanNode* root = Build_Huffman_Tree(freq);
    string huffmanCode[256];
    encodeHuffmanTree(root, "", huffmanCode);

    ofstream out(outputFile, ios::binary);
    if (!out) {
        cout << "无法创建文件: " << outputFile << endl;
        return;
    }

    // 存储频率表和比特串
    int freqCount = 0;
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) freqCount++;
    }
    out.write(reinterpret_cast<const char*>(&freqCount), sizeof(freqCount));

    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) {
            out.put(i);
            out.write(reinterpret_cast<const char*>(&freq[i]), sizeof(freq[i]));
        }
    }

    string encodedString = encodeText(text, huffmanCode);
    int bitCount = encodedString.size();
    out.write(reinterpret_cast<const char*>(&bitCount), sizeof(bitCount));

    int bitWritten = 0;
    unsigned char byte = 0;
```

```
    for (char bit : encodedString) {
        byte = (byte << 1) | (bit - '0');
        bitWritten++;
        if (bitWritten == 8) {
            out.put(byte);
            bitWritten = 0;
            byte = 0;
        }
    }
    if (bitWritten > 0) {
        byte <<= (8 - bitWritten);
        out.put(byte);
    }

    out.close();
    cout << "文件已成功压缩为: " << outputFile << endl;
}
void decompressFile(const string& inputFile, const string& outputFile) {
    ifstream in(inputFile, ios::binary);
    if (!in) {
        cout << "无法打开文件: " << inputFile << endl;
        return;
    }

    int freqCount;
    in.read(reinterpret_cast<char*>(&freqCount), sizeof(freqCount));

    int freq[256] = {0};
    for (int i = 0; i < freqCount; i++) {
        char ch;
        int frequency;
        in.get(ch);
        in.read(reinterpret_cast<char*>(&frequency), sizeof(frequency));
        freq[(unsigned char)ch] = frequency;
    }

    HuffmanNode* root = Build_Huffman_Tree(freq);

    int bitCount;
    in.read(reinterpret_cast<char*>(&bitCount), sizeof(bitCount));

    string encodedString;
    unsigned char byte;
    while (in.get((char&)byte)) {
        for (int i = 7; i >= 0; i--) {
            encodedString += ((byte >> i) & 1) ? '1' : '0';
        }
    }
    in.close();
```

```
    string decodedString = decodeText(root, encodedString, bitCount);

    ofstream out(outputFile, ios::binary);
    if (!out) {
        cout << "无法创建文件: " << outputFile << endl;
        return;
    }
    out << decodedString;
    out.close();

    cout << "文件已成功解压缩为: " << outputFile << endl;
}
```

2.4 全部代码

```
#include<iostream>
#include<string>
#include<fstream>

using namespace std;

// 哈夫曼树 节点
struct HuffmanNode {
    char ch;
    int freq;
    HuffmanNode *left, *right;
    HuffmanNode(char c, int f): ch(c), freq(f), left(nullptr), right(nullptr) {}
};

// 哈夫曼树 存放节点（比较的链表）
struct ListNode {
    HuffmanNode* huffnode;
    ListNode* next;
    ListNode(HuffmanNode* node): huffnode(node), next(nullptr) {}
};

// 插入到队列中（链表排序，按照 freq 的优先顺序： 升序）
void insert(ListNode*& head, HuffmanNode* node) {
    ListNode* newnode = new ListNode(node);
    if (!head || head->huffnode->freq > node->freq) {
        newnode->next = head;
        head = newnode;
    }
    else {
        ListNode* current =  head;
        while (current->next && current->next->huffnode->freq <= node->freq) {
```

```
            current = current->next;
        }
        newnode->next = current->next;
        current->next = newnode;
    }
}

// 从队列（链表）中取出 freq 最小的， 拿来自底向上构造 Huffman 树
HuffmanNode* PopMin(ListNode*& head) {
    if (!head) return nullptr;
    ListNode* temp = head;
    HuffmanNode* node = head->huffnode;
    head = head->next;
    delete temp;
    return node;
}

// 统计电文中 不同字符 出现的次数 freq
void calculateFrequency(int freq[], const string &text) {
    for (int i = 0; i < 256; i++) {
        freq[i] = 0;
    }
    for(char ch : text) {
        freq[(unsigned char) ch]++;
    }
}

// 构建 Huffman 树
HuffmanNode* Build_Huffman_Tree(int freq[]) {
    ListNode* head = nullptr;
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) {
            HuffmanNode* node = new HuffmanNode((char)i, freq[i]);
            insert(head, node);
        }
    }

    while(head && head->next) {
        HuffmanNode* left = PopMin(head);
        HuffmanNode* right = PopMin(head);

        int sum = left->freq + right->freq;
        HuffmanNode* combine_node = new HuffmanNode('\0', sum);
        combine_node->left = left;
        combine_node->right = right;

        insert(head,combine_node);
    }
```

```
    return PopMin(head);
}

// 对 HuffmanTree 进行 编码 left-0,  right-1, str 为编码结果
void encodeHuffmanTree(HuffmanNode* root, string str, string HuffmanCode[]) {
    if (!root) return;
    if (!root->left && !root->right) {
        HuffmanCode[(unsigned char) root->ch] = str;
    }
    else {
        encodeHuffmanTree(root->left, str + "0", HuffmanCode);
        encodeHuffmanTree(root->right, str + "1", HuffmanCode);
    }
}

// 编码输入的文本
string encodeText(const string &text, string HuffmanCode[]) {
    string encodedString;
    for (char ch : text) {
        encodedString += HuffmanCode[(unsigned char)ch];
    }
    return encodedString;
}

// 根据 Huffman 树对 比特流进行解码
string decodeText(HuffmanNode* root, const string &encodedString, int bitCount) {
    string decodedString;
    HuffmanNode* current = root;
    int bitsRead = 0;
    for (char bit : encodedString) {
        if (bitsRead >= bitCount) break;
        current = (bit == '0') ? current->left : current->right;
        bitsRead++;

        if (!current->left && !current->right) {
            decodedString += current->ch;
            current = root;
        }
    }
    return decodedString;
}

// 压缩文件
void compressFile(const string& inputFile, const string& outputFile) {
    ifstream in(inputFile, ios::binary);
    if (!in) {
        cout << "无法打开文件: " << inputFile << endl;
        return;
    }
```

```
    string text((istreambuf_iterator<char>(in)), istreambuf_iterator<char>());
    in.close();

    int freq[256] = {0};
    calculateFrequency(freq, text);

    HuffmanNode* root = Build_Huffman_Tree(freq);
    string huffmanCode[256];
    encodeHuffmanTree(root, "", huffmanCode);

    ofstream out(outputFile, ios::binary);
    if (!out) {
        cout << "无法创建文件: " << outputFile << endl;
        return;
    }

    int freqCount = 0;
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) freqCount++;
    }
    out.write(reinterpret_cast<const char*>(&freqCount), sizeof(freqCount));

    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) {
            out.put(i);
            out.write(reinterpret_cast<const char*>(&freq[i]), sizeof(freq[i]));
        }
    }

    string encodedString = encodeText(text, huffmanCode);
    int bitCount = encodedString.size();
    out.write(reinterpret_cast<const char*>(&bitCount), sizeof(bitCount));

    int bitWritten = 0;
    unsigned char byte = 0;
    for (char bit : encodedString) {
        byte = (byte << 1) | (bit - '0');
        bitWritten++;
        if (bitWritten == 8) {
            out.put(byte);
            bitWritten = 0;
            byte = 0;
        }
    }
    if (bitWritten > 0) {
        byte <<= (8 - bitWritten);
        out.put(byte);
    }
```

```
    out.close();
    cout << "文件已成功压缩为: " << outputFile << endl;
}

void decompressFile(const string& inputFile, const string& outputFile) {
    ifstream in(inputFile, ios::binary);
    if (!in) {
        cout << "无法打开文件: " << inputFile << endl;
        return;
    }

    int freqCount;
    in.read(reinterpret_cast<char*>(&freqCount), sizeof(freqCount));

    int freq[256] = {0};
    for (int i = 0; i < freqCount; i++) {
        char ch;
        int frequency;
        in.get(ch);
        in.read(reinterpret_cast<char*>(&frequency), sizeof(frequency));
        freq[(unsigned char)ch] = frequency;
    }

    HuffmanNode* root = Build_Huffman_Tree(freq);

    int bitCount;
    in.read(reinterpret_cast<char*>(&bitCount), sizeof(bitCount));

    string encodedString;
    unsigned char byte;
    while (in.get((char&)byte)) {
        for (int i = 7; i >= 0; i--) {
            encodedString += ((byte >> i) & 1) ? '1' : '0';
        }
    }
    in.close();

    string decodedString = decodeText(root, encodedString, bitCount);

    ofstream out(outputFile, ios::binary);
    if (!out) {
        cout << "无法创建文件: " << outputFile << endl;
        return;
    }
    out << decodedString;
    out.close();

    cout << "文件已成功解压缩为: " << outputFile << endl;
}
```

```
int main() {
    int choice = 1;
    while (choice) {
        cout << "请输入你的选择 1.压缩  2.解压  0.退出: ";
        cin >> choice;
        if (choice == 1) {
            string inputFilename, outputFilename;
            cout << "请输入要压缩的文件名: ";
            cin >> inputFilename;
            outputFilename = inputFilename + ".huff";
            compressFile(inputFilename, outputFilename);
        }
        else if (choice == 2) {
            string inputFilename, outputFilename;
            cout << "请输入要解压的文件名: ";
            cin >> inputFilename;
            if (inputFilename.size() > 5 && inputFilename.substr(inputFilename.size() -
5) == ".huff") {
                outputFilename = inputFilename.substr(0, inputFilename.size() - 5);
                decompressFile(inputFilename, outputFilename);
            } else {
                cout << "文件名错误" << endl;
            }
        }
        else if (choice == 0) {
            break;
        }
        else {
            cout << "选择错误! " << endl;
        }
    }
    return 0;
}
```

---

3. 调试分析

在实验过程中，程序的实现需要考虑多个因素，如Huffman树的构建、编码与解码过程的准确性、文件操作的正确性等。以下是测试数据的选择、调试中遇到的问题以及解决方法的详细分析。

3.1 测试用例

测试：文本压缩与解压

输入文件：`test.???`

压缩：

1. 输入文件`test.???`进行压缩，得到`test.txt.huff`。

解压：

1. 使用`test.txt.huff`进行解压，恢复为`test.???`。

验证：

- 解压后的文件内容应与原始文件一致。

3.2 程序调试中遇到的问题及解决方法

问题1：无法正确读取和写入二进制文件

- **问题描述**：程序在读取和写入二进制文件时，部分数据丢失或者文件损坏。
- **原因分析**：C++中，默认的文件流（`ifstream`和`ofstream`）是以文本模式打开文件的，因此可能会导致某些字符（尤其是控制字符）被错误处理，导致文件读取或写入时出现问题。
- 解决方法

  ：在读取和写入二进制文件时，必须显式地指定二进制模式。使用

  ```
  ios::binary
  ```

  标志打开文件，确保数据能够按原样读写。

  ```
  ifstream in(inputFile, ios::binary);
  ofstream out(outputFile, ios::binary);
  ```

问题2：字符频率统计不准确

- **问题描述**：程序在统计字符频率时，某些字符频率未能正确统计。
- **原因分析**：程序使用`freq[(unsigned char) ch]`来统计字符频率，如果字符是高位字符（大于127的字符），可能会导致数组越界，或者由于ASCII码的转换错误，导致某些字符没有被统计。
- 解决方法

  ：确保

  ```
  freq
  ```

  数组的大小足够容纳所有可能的字符（0-255），并且正确地处理每个字符的频率。

```
int freq[256] = {0};  // 保证频率数组有足够空间
```

问题3：Huffman编码表没有正确生成

- **问题描述**：Huffman编码生成过程中，某些字符的编码为空，或者解压后的文件与原文件不一致。
- **原因分析**：Huffman树构建过程中，可能存在某些节点没有正确连接，或者在递归编码时，某些字符没有分配到对应的编码。
- **解决方法**：
  1. 确保树的构建过程中每个节点的`left`和`right`指针正确指向左右子节点。
  2. 编码时确保遍历每个叶子节点，并将编码存储在正确的位置。

```
void encodeHuffmanTree(HuffmanNode* root, string str, string HuffmanCode[]) {
    if (!root) return;
    if (!root->left && !root->right) {
        HuffmanCode[(unsigned char) root->ch] = str;
    }
    else {
        encodeHuffmanTree(root->left, str + "0", HuffmanCode);
        encodeHuffmanTree(root->right, str + "1", HuffmanCode);
    }
}
```

问题4：压缩效率低，文件过大

- **问题描述**：对于某些输入文件，压缩后的文件大小基本不变，甚至反而比原文件大。
- **原因分析**：Huffman编码并不适用于所有类型的文本，尤其是当字符频率分布较为均匀时，Huffman编码可能会导致文件体积膨胀。另外，压缩产生的文件要存储Huffman树的信息，导致文件内容更大。
- **解决方法**：

  无，这是Huffman编码本身的局限性。

3.3 调试过程中总结

- **错误排查**：通过逐步调试和打印输出，分析每一部分的执行过程，发现并修正了文件读取、频率统计、树的构建等模块中的问题。
- **代码重构**：在过程中，发现了多个潜在的代码优化点，例如改进频率统计、优化节点插入和链表操作等，使程序更高效。
- **单元测试**：通过不同的数据集进行多轮测试，确保程序能够在各种情况下正常工作。

4. 复杂度分析

4.1 时间复杂度

1. **构建Huffman树**：构建过程涉及到插入和弹出操作，每个操作的时间复杂度为O(log N)，其中N是树中的节点数量。总的时间复杂度为O(N log N)。
2. **编码与解码**：编码和解码的时间复杂度为O(M)，其中M是文件的总字符数。

4.2 空间复杂度

1. **Huffman树**：树结构的空间复杂度为O(N)，其中N是树的节点数。
2. **频率数组和编码表**：频率数组的空间复杂度为O(256)，编码表的空间复杂度为O(256)。

因此，总体空间复杂度为O(N + M)，N为树的节点数，M为文件大小。

5. 总结与体会

本实验通过实现Huffman编码的压缩与解压缩过程，深入理解了Huffman算法的基本原理及其在数据压缩中的应用。实验中遇到的一些问题，如编码表的构建、比特流的处理等，帮助我加深了对数据压缩算法的理解。此外，通过实现文件操作的相关功能，我提升了自己在文件处理和算法设计方面的能力。

Lab4 实验报告

实验报告：基于邻接表的Dijkstra算法实现最短路径

1. 问题的描述

1.1 设计目标

本次实验的目标是实现Dijkstra算法，使用邻接表存储图，并手动实现一个优先队列来优化Dijkstra算法的性能。该算法用于解决单源最短路径问题，给定一个有权图和起始节点，计算从起始节点到目标节点的最短路径权重。如果存在路径，则输出最短路径的权重，否则输出没有路径。

1.2 输入数据

- 图的节点数`n`，边数`m`。
- `m`条边，每条边包含三个整数`a`, `b`, `w`，表示从节点`a`到节点`b`的边，权重为`w`。
- 起始节点`s`和目标节点`t`。

1.3 输出结果

- 输出从起始节点`s`到目标节点`t`的最短路径的总权重。如果节点`t`不可达，则输出`No path found`。

---

2. 算法的描述

2.1 数据结构的描述

逻辑结构

- **邻接表**：使用邻接表表示图，`graph[u]`包含一个指向节点`u`的所有邻接边的链表，每条边包含终点`v`和权重`w`。
- **优先队列**：使用一个简单的手动实现的优先队列（基于数组），用于存储节点和其当前最短路径权重。每次选择最小路径权重的节点进行松弛操作。

存储结构（直接从作业题中挪用）

- **Graph结构**：包含`numVertex`（节点数）和`adjList`（邻接表数组）。`adjList`的每个元素是一个链表头指针，链表节点`AdjNode`包含`vertex`（目标节点）和`weight`（边的权重）。

```
struct AdjNode {
    int vertex, weight;
    AdjNode* next;
    AdjNode(int v, int w) : vertex(v), weight(w), next(nullptr) {}
};
```

- **PriorityQueue结构**：手动实现的优先队列，包含一个`heap`数组用于存储节点，`distance`数组用于存储每个节点的当前最短路径权重，`size`表示队列的大小。

```
struct PriorityQueue {
    int* heap;
    int* distance;
    int size;
    void push(int vertex, int dist);
    int pop();
    bool empty();
};
```

- **`dist[]`数组**：存储从起始节点到每个节点的最短路径的当前权重。
- **`visited[]`数组**：标记每个节点是否已经被访问过。

2.2 程序结构的描述

函数原型

- **图的结构与加边（直接从作业题中挪用）** `void Graph::addEdge(int src, int dest, int weight)`：
  - 功能：向图中添加一条边`src -> dest`，并且因为是无向图，还需添加`dest -> src`的反向边。
  - 输入：边的起点`src`，终点`dest`，权重`weight`。
- **Dijkstra算法** `int dijkstra(Graph& graph, int start, int end)`：
  - 功能：使用Dijkstra算法计算从节点`start`到节点`end`的最短路径。
  - 输入：图`graph`，起始节点`start`，目标节点`end`。
  - 输出：从`start`到`end`的最短路径权重。
- `PriorityQueue::push(int vertex, int dist)`：
  - 功能：向优先队列中插入一个节点`vertex`和其对应的最短路径权重`dist`。
  - 输入：节点`vertex`，权重`dist`。
- `PriorityQueue::pop()`：
  - 功能：从优先队列中弹出权重最小的节点并返回。
  - 输出：权重最小的节点。
- `PriorityQueue::empty()`：
  - 功能：判断优先队列是否为空。
  - 输出：若队列为空，返回`true`，否则返回`false`。

2.3 全部代码

```
#include<iostream>
#include <climits>
using namespace std;

// 邻接表节点
struct AdjNode {
    int vertex, weight;
    AdjNode* next;
    AdjNode(int v, int w) : vertex(v), weight(w), next(nullptr) {}
};

// 图的结构
struct Graph {
    int numVertex;
    AdjNode** adjList;

    Graph(int vertices) {
        numVertex = vertices;
```

```
        adjList = new AdjNode*[vertices + 1]; // 1-based indexing
        for (int i = 0; i <= numVertex; i++) {
            adjList[i] = nullptr;
        }
    }

    // 添加边
    void addEdge(int src, int dest, int weight) {
        // 添加 src -> dest
        AdjNode* newNode = new AdjNode(dest, weight);
        newNode->next = adjList[src];
        adjList[src] = newNode;

        // 添加 dest -> src (无向图)
        newNode = new AdjNode(src, weight);
        newNode->next = adjList[dest];
        adjList[dest] = newNode;
    }

};
// 手动实现 优先队列
struct PriorityQueue {
    int* heap;
    int* distance;
    int size;

    PriorityQueue(int n) {
        heap = new int[n + 1];
        distance = new int[n + 1];
        size = 0;
    }

    void push(int vertex, int dist) {
        heap[++size] = vertex;
        distance[vertex] = dist;
    }

    int pop() {
        int minVertex = -1, minDist = INT_MAX, minIndex = -1;
        for (int i = 1; i <= size; ++i) {
            if (distance[heap[i]] < minDist) {
                minDist = distance[heap[i]];
                minVertex = heap[i];
                minIndex = i;
            }
        }
        if (minIndex != -1) {
            // 移除堆顶元素
            heap[minIndex] = heap[size--];
```

```
        }
        return minVertex;
    }

    bool empty() {
        return size == 0;
    }
};

// Dijkstra 算法
int dijkstra(Graph& graph, int start, int end) {
    int* dist = new int[graph.numVertex + 1];
    bool* visited = new bool[graph.numVertex + 1];

    // 初始化
    for (int i = 1; i <= graph.numVertex; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }
    dist[start] = 0;

    // 优先队列
    PriorityQueue pq(graph.numVertex);
    pq.push(start, 0);

    while (!pq.empty()) {
        int u = pq.pop();
        if (visited[u]) continue;
        visited[u] = true;

        // 遍历邻接表
        AdjNode* neighbor = graph.adjList[u];
        while (neighbor != nullptr) {
            int v = neighbor->vertex;
            int weight = neighbor->weight;

            if (!visited[v] && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push(v, dist[v]);
            }
            neighbor = neighbor->next;
        }
    }

    int result = dist[end];
    delete[] dist;
    delete[] visited;

    return result == INT_MAX ? -1 : result;
```

```
}

int main() {
    int n, m;
    cin >> n >> m;

    Graph graph(n);

    for (int i = 0; i < m; i++) {
        int a, b, w;
        cin >> a >> b >> w;
        graph.addEdge(a, b, w);
    }

    int s, t;
    cin >> s >> t;

    int result = dijkstra(graph, s, t);
    if (result == -1) {
        cout << "No path found" << endl;
    } else {
        cout << result << endl;
    }

    return 0;
}
```

---

3. 调试分析

3.1 程序调试中遇到的问题及解决方法

问题1：优先队列的实现问题

- **描述**：在手动实现优先队列时，发现优先队列中的节点没有正确地按照最小权值排序，导致Dijkstra算法未能正确地选择最小的节点进行松弛操作。
- **解决方法**：修改了`pop()`函数，通过遍历整个队列找到最小权值节点，并移除它。

问题2：邻接表的构建错误

- **描述**：在添加边时，邻接表的插入顺序不对，导致图的结构错误，影响结果。
- **解决方法**：确保每条边在邻接表中正确地反向插入，保证无向图的对称性。

问题3：Dijkstra的初始化问题

- **描述**：在初始化`dist[]`数组时，未正确处理源节点的最短路径，导致结果错误。
- **解决方法**：确保在初始化时，将起始节点的`dist[start]`设置为0，其他节点设置为`INF`。

---

4. 算法的时空分析

4.1 时间复杂度

- **Dijkstra算法**：每次从优先队列中弹出一个节点，遍历该节点的邻接节点并更新最短路径。对每个节点，最多需要遍历其所有邻接边。因此，时间复杂度为`O((V + E) log V)`，其中`V`为节点数，`E`为边数。由于优先队列的每次插入和删除操作是`O(log V)`。
- **手动实现的优先队列**：每次从队列中选出最小值的操作是`O(V)`，而`V`是当前节点数，因此优化空间较大。实际应用中可以使用更高效的优先队列（如二叉堆或Fibonacci堆）。

4.2 空间复杂度

- **邻接表**：空间复杂度为`O(V + E)`，其中`V`是节点数，`E`是边数。
- **优先队列**：空间复杂度为`O(V)`，用于存储所有节点及其对应的最短路径。

---

5. 测试结果及分析

5.1 测试数据1

输入：

```
4 5
1 2 10
1 3 20
2 3 15
2 4 30
3 4 20
1 4
```

输出：

```
40
```

分析：

- 从节点1到节点4的最短路径为`1 -> 2 -> 4`或`1 -> 3 -> 4`，总权重为40。程序正确计算了最短路径。

---

6. 实验体会和收获

通过本次实验，我深入理解了Dijkstra算法的原理和实现，掌握了如何使用邻接表存储图以及如何手动实现优先队列来优化算法性能。通过解决实际中的调试问题，我加深了对图算法和数据结构的理解，特别是在图的表示和最短路径计算的细节方面。

实验报告：图的深度优先遍历（DFS）与广度优先遍历（BFS）

1. 问题的描述

设计目标：设计一个图的遍历程序，能够对给定的图进行深度优先遍历（DFS）和广度优先遍历（BFS）。图采用邻接矩阵表示，程序要求从指定的起始节点出发，分别输出深度优先和广度优先的遍历顺序。

输入数据：

- 图的节点数`V`，边数`E`。
- 接下来输入`E`条边，表示图中节点间的连接关系。
- 一个起始节点`start`，表示遍历从此节点开始。

输出结果：

- 程序分别输出从`start`节点出发的深度优先遍历（DFS）和广度优先遍历（BFS）顺序。

---

2. 算法的描述

(1) 数据结构的描述

- **逻辑结构**：采用图的邻接矩阵来存储图的结构，矩阵`graph[i][j]`表示节点`i`与节点`j`是否相邻（若相邻则为`1`，否则为`0`）。
- 存储结构
  :
  - `graph[MAXV + 1][MAXV + 1]`：邻接矩阵，用来存储图中节点之间的连接信息，大小为`(MAXV + 1) * (MAXV + 1)`，最大支持`MAXV = 30`个节点。
  - `visited[MAXV + 1]`：访问数组，用于记录每个节点是否已经被访问，大小为`MAXV + 1`，避免节点重复访问。

主要变量说明：

- `graph[i][j]`：表示节点`i`与节点`j`之间是否有边，若有边则为`1`，否则为`0`。
- `visited[i]`：若节点`i`已被访问，则`visited[i]`为`1`，否则为`0`。
- `dfsresult[MAXV]`、`bfsresult[MAXV]`：用于存储深度优先遍历和广度优先遍历的结果。
- `dfsindex`、`bfsindex`：分别记录DFS和BFS结果数组的当前下标，用于依次填充遍历顺序。

(2) 程序结构的描述

函数原型：

- `void initializeArray(int arr[], int size, int value)`：初始化数组`arr`，将其所有元素设置为`value`。
- `void initializeMatrix(int Mat[][MAXV + 1], int size, int value)`：初始化邻接矩阵`Mat`，将其所有元素设置为`value`。
- `void dfs(int node, int n, int result[], int& index)`：深度优先遍历函数，从`node`开始，遍历图中的所有节点，结果存储在`result`数组中。
- `void bfs(int start, int n, int result[], int& index)`：广度优先遍历函数，从`start`开始，遍历图中的所有节点，结果存储在`result`数组中。
- `int main()`：主函数，负责输入图的数据，调用DFS和BFS函数，并输出遍历结果。

功能描述：

- **DFS函数**：
  - 功能：从指定的节点`node`开始，递归地进行深度优先遍历。
  - 输入：当前节点`node`，节点总数`n`，结果数组`result`，当前遍历结果的索引`index`。
  - 输出：将深度优先遍历的节点按顺序存储到`result`数组中。
- **BFS函数**：
  - 功能：从指定的起始节点`start`开始，按照广度优先的顺序遍历图。
  - 输入：起始节点`start`，节点总数`n`，结果数组`result`，当前遍历结果的索引`index`。
  - 输出：将广度优先遍历的节点按顺序存储到`result`数组中。

（3）全部代码

```
#include<iostream>
using namespace std;

const int MAXV = 30;
const int MAXE = 300;

// 邻接矩阵
int graph[MAXV + 1][MAXV + 1];
// 辅助访问数组
int visited[MAXV + 1];

// 初始化数组
void initializeArray(int arr[], int size, int value) {
    for (int i = 0; i < size; i++) {
        arr[i] = value;
    }
}
```

```
void initializeMatrix(int Mat[][MAXV + 1], int size, int value) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            Mat[i][j] = value;
        }
    }
}

void dfs(int node, int n, int result[], int& index) {
    visited[node] = 1;  // 本节点已访问
    result[index++] = node;

    // 依次递归dfs node 的所有邻接节点
    for (int i = 1; i <= n; i++) {
        if (graph[node][i] && !visited[i]) {
            dfs(i, n, result, index);
        }
    }
}

void bfs(int start, int n, int result[], int& index) {
    int quene[MAXV + 1]; // 模拟队列
    int front = 0, rear = 0;

    quene[rear++] = start; // 访问入队
    visited[start] = 1;

    while (rear > front) {
        int node = quene[front++];  // 取出队首元素
        result[index++] = node;     // 记录访问序列

        // 依次遍历邻接节点
        for (int i = 1; i <= n; i++) {
            if (graph[node][i] && !visited[i]) {

                quene[rear++] = i;  // 入队
                visited[i] = 1;     // 标记为已经访问
            }
        }
    }
}

int main() {
    int V, E;
    cin >> V >> E;

    initializeArray(visited, MAXV + 1, 0);
    initializeMatrix(graph,MAXV + 1, 0);
```

```
    // 输入，构建矩阵图
    for (int i = 0; i < E; i++) {
        int a, b;
        cin >> a >> b;
        graph[a][b] = graph[b][a] = 1;
    }

    int start;
    cin >> start;

    // dfs遍历
    int dfsresult[MAXV], dfsindex = 0;
    dfs(start, V, dfsresult, dfsindex);

    // 输出
    for (int i = 0; i < dfsindex; i++) {
        if(i > 0) {
            cout << " ";
        }
        cout << dfsresult[i];
    }
    cout << endl;

    // 再次初始化
    initializeArray(visited, MAXV + 1, 0);

    // bfs
    int bfsresult[MAXV], bfsindex = 0;
    bfs(start, V, bfsresult, bfsindex);
    // 输出
    for (int i = 0; i < bfsindex; i++) {
        if(i > 0) {
            cout << " ";
        }
        cout << bfsresult[i];
    }
    cout << endl;

    return 0;

}
```

3. 调试分析

测试数据的选择：

为了全面测试程序的正确性，我们选择了不同结构的图进行测试：

- **测试数据1**：一个简单的无向图，包含5个节点和6条边。用于验证基本功能。
- **测试数据2**：一个包含孤立节点的图，验证程序是否能够正确处理未连接的节点。

程序调试中遇到的问题及解决方法：

1. **问题1：邻接矩阵初始化错误**
   - **描述**：程序在初始化邻接矩阵时没有正确设置矩阵的大小，导致部分边未能正确录入。
   - **解决方法**：通过调用`initializeMatrix`函数，确保邻接矩阵被初始化为零，避免错误的边信息影响后续计算。
2. **问题2：访问数组未初始化**
   - **描述**：在DFS和BFS中，访问数组未正确初始化，导致部分节点被重复访问。
   - **解决方法**：在每次遍历之前，都调用`initializeArray`函数初始化访问数组，确保每次遍历从头开始。
3. **问题3：输出格式问题**
   - **描述**：输出结果时，节点之间缺少空格，导致输出格式不符合要求。
   - **解决方法**：修改输出部分代码，确保每个节点之间有空格，且每个遍历的结果在行尾添加换行符。

---

4. 算法的时空分析

时间复杂度：

- **DFS**：对于每个节点，我们最多访问一次所有邻接的节点。时间复杂度为`O(V + E)`，其中`V`为节点数，`E`为边数。
- **BFS**：每个节点被访问一次，每条边也被访问一次。时间复杂度同样为`O(V + E)`。

空间复杂度：

- **DFS**：DFS使用递归栈，最大递归深度为`V`，因此空间复杂度为`O(V)`。
- **BFS**：BFS使用队列存储待访问的节点，最大队列大小为`V`，因此空间复杂度为`O(V)`。

---

5. 测试结果及分析

测试数据1：

```
输入：
5 6
1 5
1 4
3 1
2 1
2 5
5 3
1

输出：
DFS遍历：1 2 5 3 4
BFS遍历：1 2 3 4 5
```

测试数据2：

```
输入：
4 2
1 2
2 3
1

输出：
DFS遍历：1 2 3
BFS遍历：1 2 3
```

分析：

- 测试数据1验证了程序在一般情况下的功能，DFS和BFS遍历顺序正确。
- 测试数据2验证了程序在处理部分孤立节点时的正确性，程序能够正确处理并输出正确的遍历顺序。

---

6. 实验体会和收获

通过本次实验，我深入理解了图的深度优先遍历和广度优先遍历的原理及实现方法。通过邻接矩阵表示图的结构，掌握了如何在程序中使用递归和队列来实现这两种遍历算法。同时，我也学会了如何进行图的调试和优化，确保程序在不同测试数据下都能正常工作。

实验过程中，我还遇到了数组初始化和输出格式的问题，通过解决这些问题，增强了我对程序细节的关注。总体来说，本次实验帮助我加深了对图论基础算法的理解，并提高了编程和调试的能力。

Lab5 实验报告

哈希表实验报告

1. 问题的描述

本实验的目的是实现两种常见的哈希表结构：线性探测哈希表和链表哈希表，并进行测试。哈希表用于解决冲突问题，允许快速查找、插入和删除操作。实验的目标是：

- 实现线性探测和链表哈希表。
- 使用哈希函数对给定的关键字进行哈希映射。
- 支持插入操作和查找操作，并计算相应的查找次数。
- 比较不同哈希表结构在查找性能上的差异。

输入数据：

1. 哈希表的大小（表的大小）。
2. 哈希函数的模数（p）。
3. 关键字数量（n），以及n个要插入的关键字。

输出结果：

1. 哈希表的状态，显示每个槽位的内容。
2. 每个关键字的查找成功次数与失败次数。
3. 查找成功和失败的平均查找长度。

2. 算法的描述

(1) 数据结构的描述

线性探测哈希表

- **存储结构**：使用一个一维数组 `table[]` 存储哈希表的内容，另外用一个布尔型数组 `occupied[]` 来标记每个槽是否被占用。
- **主要变量**：
  - `table[]`：存储插入的关键字。
  - `occupied[]`：标记每个槽是否已经被占用，避免重复插入。
  - `size`：哈希表的大小。
  - `p`：哈希函数的模数，用于确定插入位置。

链表哈希表

- **存储结构**：使用一个指向节点结构体的数组 `table[]`。每个槽位是一个链表，链表节点包含一个关键字以及指向下一个节点的指针。
- **主要变量**：

- `table[]`：存储链表头指针的数组。
- `Node` 结构体：每个节点包含关键字 `key` 和指向下一个节点的指针 `next`。
- `size`：哈希表的大小。
- `p`：哈希函数的模数，用于确定插入位置。

(2) 程序结构的描述

函数原型与功能

- `int hashfunction(int key, int p)`：哈希函数，输入一个关键字 `key` 和模数 `p`，返回哈希值，即 `key % p`。
- **HashTableLinear 类**：
  - `insert(int key)`：插入一个关键字到哈希表中，使用线性探测解决冲突。
  - `search(int key, bool& success)`：查找一个关键字，返回查找是否成功以及探测次数。
  - `printTable()`：打印哈希表的内容以及每个槽的探测次数。
- **HashTableChaining 类**：
  - `insert(int key)`：插入一个关键字到哈希表中，使用链表解决冲突。
  - `search(int key, bool& success)`：查找一个关键字，返回查找是否成功以及探测次数。
  - `printTable()`：打印哈希表的内容和每个槽的探测次数。

全部代码：

```
#include <iostream>
using namespace std;

// 哈希函数
int hashfunction(int key, int p) {
    return key % p;
}

// 线性探测哈希表
class HashTableLinear {
private:
    int* table;      // 存储数据
    bool* occupied; // 当前位置是否被占用
    int size;        // 表的大小
    int p;           // 哈希表的模数

public:
    // 构造函数
    HashTableLinear(int TableSize, int mod): size(TableSize), p(mod) {
        table = new int[size];
        occupied = new bool[size];
```

```
        // 初始化哈希表
        for (int i = 0; i < size; i++) {
            table[i] = -1;          // -1 表示空槽
            occupied[i] = false;
        }
    }

    // 插入元素
    void insert(int key) {
        int index = hashfunction(key, p);
        while (occupied[index]) {
            index = (index + 1) % size;
        }
        table[index] = key;
        occupied[index] = true;
    }

    // 查找元素，返回成功与否以及探测次数
    int search(int key, bool& success) {
        int index = hashfunction(key, p);
        int probes = 0;
        while (probes < size) {
            probes++;
            if (!occupied[index]) {
                success = false;
                return probes; // 查找失败
            }
            if (table[index] == key) {
                success = true;
                return probes; // 查找成功
            }
            index = (index + 1) % size;
        }
        success = false;
        return probes;
    }

    // 计算失败查找次数：对于有关键字的失败查找，查找会遇到空槽
    int getFailureProbes(int key) {
        int index = hashfunction(key, p);
        int probes = 0;
        while (probes < size) {
            probes++;
            if (!occupied[index]) {
                return probes;  // 遇到空槽，返回失败探测次数
            }
            index = (index + 1) % size;
        }
        return probes;
```

```
    }

    // 打印哈希表并计算探测次数
    void printTable() {
        cout << "哈希表的地址:\t";
        for (int i = 0; i < size; i++) {
            cout << i << "\t";
        }
        cout << endl;

        cout << "表中的关键字:\t";
        for (int i = 0; i < size; i++) {
            if (occupied[i]) {
                cout << table[i] << "\t";
            } else {
                cout << "-\t";
            }
        }
        cout << endl;

        // 初始化成功和失败查找次数数组
        int* successProbes = new int[size]();
        int* failureProbes = new int[size]();

        // 计算每个元素的成功查找次数
        for (int i = 0; i < size; i++) {
            if (occupied[i]) {
                bool success;
                successProbes[i] = search(table[i], success);
            }
        }

        // 计算每个元素的失败查找次数
        for (int i = 0; i < size; i++) {
            if (occupied[i]) {
                failureProbes[i] = size - i - 1;
            } else {
                // 空槽的失败查找次数为 1
                failureProbes[i] = 1;
            }
        }

        // 输出探测次数
        cout << "成功查找次数:\t";
        for (int i = 0; i < size; i++) {
            cout << successProbes[i] << "\t";
        }
        cout << endl;
```

```
        cout << "失败查找次数:\t";
        for (int i = 0; i < size; i++) {
            cout << failureProbes[i] << "\t";
        }
        cout << endl;

        // 计算平均查找长度
        double successTotal = 0, failureTotal = 0;
        int successCount = 0, failureCount = 0;

        for (int i = 0; i < size; i++) {
            if (occupied[i]) {
                successTotal += successProbes[i];
                successCount++;
            }
            failureTotal += failureProbes[i];
            failureCount++;
        }

        cout << "查找成功的平均查找长度: "
             << (successCount > 0 ? successTotal / successCount : 0) << endl;
        cout << "查找失败的平均查找长度: "
             << (failureCount > 0 ? failureTotal / failureCount : 0) << endl;

        delete[] successProbes;
        delete[] failureProbes;
    }
};

// 链表Hash
class HashTableChaining {
private:
    struct Node {
        int key;
        Node* next;
        Node(int k): key(k), next(nullptr) {}
    };

    Node** table;
    int size;
    int p;

public:
    HashTableChaining(int TableSize, int mod): size(TableSize), p(mod) {
        table = new Node*[size];
        for (int i = 0; i < size; i++) {
            table[i] = nullptr;
        }
    }
```

```
void insert(int key) {
    int index = hashfunction(key, p);
    Node* newNode = new Node(key);
    newNode->next = table[index];
    table[index] = newNode;

}

int search(int key, bool& success) {
    int index = hashfunction(key, p);
    Node* current = table[index];
    int probes = 0;
    while (probes < size) {
        probes++;
        if (!current) {
            success = false;
            return probes;
        }
        if (current->key == key) {
            success = true;
            return probes;
        }
        current = current->next;

    }
    success = false;
    return probes;
}
void printTable() {
    cout << "哈希表的地址:\t";
    cout << "表中的关键字:\t";
    cout << endl;
    for (int i = 0; i < size; i++) {
        cout << i << "\t";
        Node* current = table[i];
        if (current) {
            while (current) {
                cout << current->key << " ";
                current = current->next;
            }
            cout << "\t";
        } else {
            cout << "-\t";
        }
        cout << endl;
    }
    cout << endl;
```

```
        // 初始化探测次数数组
        int* successProbes = new int[size];
        int* failureProbes = new int[size];
        for (int i = 0; i < size; i++) {
            successProbes[i] = 0; // 初始化为 0
            failureProbes[i] = 0; // 初始化为 0
        }

        // 计算每个槽位的成功和失败查找次数
        for (int i = 0; i < size; i++) {
            Node* current = table[i];
            int chainLength = 0; // 当前槽位链表长度
            while (current) {
                chainLength++;
                cout << current->key << "对应成功查找次数:\t" << chainLength << endl;
                successProbes[i] += chainLength; // 累加成功查找的探测次数

                current = current->next;
        }
            failureProbes[i] = chainLength + 1; // 失败查找的探测次数 = 链表长度 + 1
        }
        cout << endl;
        cout << "失败查找次数:\t";
        for (int i = 0; i < size; i++) {
        cout << failureProbes[i] << "\t";
    }
        cout << endl;

        // 计算成功和失败的平均查找长度
        double successTotal = 0, failureTotal = 0;
        int successCount = 0, failureCount = size;

        for (int i = 0; i < size; i++) {
        if (table[i]) {
            successTotal += successProbes[i];
            Node* current = table[i];
            while (current) {
                successCount++; // 统计总的成功查找数
                current = current->next;
            }
        }
        failureTotal += failureProbes[i];
    }

        cout << "查找成功的平均查找长度: " << (successCount > 0 ? successTotal /
successCount : 0) << endl;
        cout << "查找失败的平均查找长度: " << (failureCount > 0 ? failureTotal /
failureCount : 0) << endl;
```

```
        delete[] successProbes;
        delete[] failureProbes;
}

};


// 主函数
int main() {
    int tableSize, mod, numKeys;
    cout << "请输入哈希表大小: ";
    cin >> tableSize;
    cout << "请输入模数: ";
    cin >> mod;
    cout << "请输入关键字数量: ";
    cin >> numKeys;

    HashTableLinear hashTable1(tableSize, mod);
    HashTableChaining hashTable2(tableSize, mod);

    cout << "请输入关键字: ";
    for (int i = 0; i < numKeys; i++) {
        int key;
        cin >> key;
        hashTable1.insert(key);
        hashTable2.insert(key);
    }

    hashTable1.printTable();
    hashTable2.printTable();
    return 0;
}
```

---

3. 调试分析

程序调试中遇到的问题及解决方法

1. **线性探测中的循环问题**：
    - 问题：在使用线性探测时，数组的访问超出了表的大小，导致访问非法内存。
    - 解决方法：在探测时使用模运算，确保索引始终在表的大小范围内。
2. **链表哈希表中的链表操作问题**：
    - 问题：链表插入时，未正确更新链表头指针，导致新插入的元素丢失。
    - 解决方法：确保每次插入时，新的节点被插入到链表的头部，并更新链表头指针。

3. **输出问题**；
    - 问题： 链式哈希表 难以以一目了然的输出格式进行输出
    - 解决方法： 参考输出样例，将输出格式进行转置，使得哈希表地址及其对应的关键字输出更加美观。

4. 算法的时空分析

时间复杂度

1. **线性探测哈希表**：
    - **插入操作**：最坏情况下，插入的时间复杂度是 O(n)，其中 n 是哈希表的大小，发生在哈希冲突较多的情况下。
    - **查找操作**：最坏情况下，查找的时间复杂度是 O(n)，因为可能需要遍历整个哈希表。
2. **链表哈希表**：
    - **插入操作**：每个链表的插入操作时间复杂度是 O(1)，即使发生哈希冲突，也只需要在链表头部插入。
    - **查找操作**：最坏情况下，查找的时间复杂度是 O(n)，当所有元素都哈希到同一槽位时，链表的查找复杂度为 O(n)。

空间复杂度

1. **线性探测哈希表**：
    - **空间复杂度**：O(n)，需要一个大小为 n 的数组来存储哈希表的数据和一个大小为 n 的布尔数组来标记占用情况。
2. **链表哈希表**：
    - **空间复杂度**：O(n)，每个槽位是一个链表，链表节点需要额外的空间来存储关键字和指向下一个节点的指针。

5. 测试结果及分析

测试数据

输入数据：

```
请输入哈希表大小： 11
请输入模数： 11
请输入关键字数量： 8
请输入关键字： 23 35 12 56 123 39 342 90
```

输出：

线性探测哈希表

```
哈希表的地址:   0       1       2       3       4       5       6       7       8       9       10
表中的关键字:   -       23      35      12      56      123     39      342     90      -       -
成功查找次数:   0       1       1       3       4       4       1       7       7       0       0
失败查找次数:   1       9       8       7       6       5       4       3       2       1       1
查找成功的平均查找长度: 3.5
查找失败的平均查找长度: 4.27273
```

链表哈希表

```
哈希表的地址:   表中的关键字:
0       -
1       342 56 12 23
2       90 123 35
3       -
4       -
5       -
6       39
7       -
8       -
9       -
10      -

342对应成功查找次数:    1
56对应成功查找次数:     2
12对应成功查找次数:     3
23对应成功查找次数:     4
90对应成功查找次数:     1
123对应成功查找次数:    2
35对应成功查找次数:     3
39对应成功查找次数:     1

失败查找次数:   1       5       4       1       1       1       2       1       1       1       1
查找成功的平均查找长度: 2.125
查找失败的平均查找长度: 1.72727
```

分析

- 线性探测哈希表在插入元素时，当发生哈希冲突时，需要不断探测其他槽位，导致查找的平均长度增加。
- 链表哈希表通过链表解决冲突，因此查找元素时需要遍历链表，但链表的长度一般较短，查找成功的平均查找长度较小。

6. 实验体会和收获

通过本次实验，我深刻理解了哈希表的两种冲突解决策略：线性探测和链表哈希。线性探测的简单实现使得它在内存上更紧凑，但在处理大量冲突时效率较低。而链表哈希则通过使用链表存储冲突的元素，避免了连续探测的开销，但增加了内存开销。实验加深了我对哈希表效率分析的理解，尤其是不同负载因子对性能的影响。